# Contextualized: MVP

## Purpose

The purpose of this documentation is to separate our features that are considered “nice-to-have” versus features that are a part of our MVP (minimum viable product). The output of this documentation should be the separation between these features and why these decisions are being made. By doing so, the hope is that we will have cemented our plan to move forward with the remainder of our project while also being able to have something we can expand on in the future if we so choose.

## Current Progress

As of this point (1/2/2025), the main logic that we have already completed is as follows:

1. Creating a **Project**
    a. Basic functionality, really just a way to “group” data sources
2. Creating a **DataSource**
    a. As of now, only valid DataSource provider that we can account for is **GitHub**
3. Creating an IngestionJob
    a. Asynchronous capabilities (kicking off job runs in background)
    b. Executing long running processes (i.e chunking of ingested documentation) in a separate worker thread
    c. Again, only able to ingest from **GitHub** currently
    d. Code & documentation correctly downloaded, chunked intelligently, and stored within our vector database via embedding configured for particular project

## Brain Dump of Features

In this section, we will simply list all the capabilities that are currently top of mind surrounding our project **Contextualized.** There will be no inherent order to this list, simply just a dumping of all things we have considered to be a worth\while feature that this project should implement.

1. **Unit Tests**
    a. Leveraging unit tests to verify the core features of our application are working as expected
2. **Creating Conversation & Message Entities**
    a. Creating relevant logic for a user to create a conversation and subsequently send a Message to LLM (and LLM to send a message back to user)
3. **Exception Handlers**

    a. Configure some custom exceptions and some Controller Advice for handling these specific exceptions being thrown, ensuring our frontend will have descriptive information regarding the failure
4. **CORS & JWT Middleware**
    a. Ensure that our CORS is configured for frontend communication
    b. Introduce JWT filtering capabilities to ensure user making request is “authenticated” and able to hit the particular endpoint
5. **Authentication, User, and Team Management**
    a. Introduce “user” aspect that allows for configurations to be a on “per-user” basis rather than on a “global” basis
    b. Introduce logic for a user to “onboard” to a team
    c. Setup permissions for teams to be able to view certain projects
    d. Allow for user configuration such as which LLM to use, which projects are pinned, etc
    e. Admin-only logic
6. **LLM Configuration**
    a. Helper functions for basic LLM capabilities
        i. Tokenizing user input
        ii. Checking if “selected” LLM is available (and self-healing if not)
        iii. Determining context length of the model selected and taking into account any local system restrictions (if users running locally) such as no GPU, amount of VRAM/RAM, etc
        iv. Decomposition of users questions (breaking down complex questions into multiple questions)
7. **RAG Pipeline**
    a. Configuring necessary logic to integrate our RAG pipeline into basic user conversations
    b. Not querying Chroma DB every single time no matter what was said (i.e clarifying questions should not just automatically requery database)
    c. Previous pieces of conversation should be included in the context of message
8. **Additional Data Providers**
    a. Allow for capabilities such as Bitbucket, Confluence, etc to be “connected”
    b. This will allow for users of our application to get information they can use RAG pipeline across many different locations
9. **Cleanup Stale Files from Chroma DB**
    a. We should ensure that when we fail to see a file that was seen previously during ingestion job, and remove from the relational db, that we also remove the file from Chroma DB (ensuring that there is no duplication of files or extraneous code that no longer applies)
10. **Optimizing Ingestion Job Performance**
    a. Determine if there's any parallelization that we could leverage while ingesting code or splitting up logic into “batches” for larger ingestion jobs (essentially making it more efficient and toying around with performance)

    b. Account for large files when we download (we currently read entire file into memory)
11. **Multi-Modal Chat Capabilities**
    a. Allow for images to be sent directly when conversing with models (i.e screenshots of ideas that they may have, architecture diagrams)
12. **Handling Private Repositories or Data Providers**
    a. Dealing with GitHub private repositories
    b. Requiring of API keys for Confluence, etc
13. **Jira Integration**
    a. Being able to filter out pieces of code that pertains to a particular Jira ticket number (or, even better, all Jira Ticket Numbers under a particular ticket)
    b. This allows for code to be specific to a project (i.e only seeing code that your team worked on)
14. **Logging Expansion**
    a. Using thread IDs in each log, helping to distinguish logs from two separate invocations of our flows
15. **Project Logic & Architecture Expansion**
    a. Setup dependencies between projects
    b. Creating ChromaDB for architecture decisions (i.e connecting multiple systems together)
16. **Refactorings**
    a. Separating constants from configs
    b. Separate dev.test from production dependencies
    c. Combining similar functions (i.e chunking of files)
17. **Query Decomposition & Chroma Collection Suggestion**
    a. Decompose user query into separate queries if the question is dense
    b. Use LLM to decompose query into multiple queries
    c. Use LLM to determine which chroma DB we should query from

# MVP Features

In this section, we will try to list out the features that we are deeming as MVP to **Version 1.0** of our project. This doesn't necessarily mean all features left off of this are not going to be added, but I believe we need to see a goal on the horizon that we strive towards in an efficient manner without getting caught up in scope-creep.

**Note:** The following list will be in "highest priority" order in order to ensure that we have a high level plan that we can reference back to while continuing forward with this project.

1. **Unit Tests for Ingestion Job Logic**

    a. Utilize **Antigravity** to help write some unit tests for the **core logic** of our ingestion job
    b. Don't get too concerned with code coverage, let's just make sure that basic principles that are the root of our ingestion logic are working as expected
2. **Setup Message Entity**
    a. Create the tables required for storing information about a Message
    b. See **todo.txt** for relevant attributes to be included
3. **Setup Basic Endpoints for Querying a Project**
    a. This should just be a **one-and-done** style approach of just simply getting a clear and concise answer back from LLM
    b. No previous conversation history logic, just simply Q&A that we can build off in future
4. **LLM Functionality**
    a. We should be able to do the following with configured LLM
        i. Determine which LLM the user selected
        ii. Setup so that it can be leveraged by LlamaIndex when we query the RAG
        iii. Setup "system" prompt
        **iv.** **IF LOCAL LLM** → validate that LLM installed (if not, prompt user to install via ollama)
5. **RAG Pipeline**
    a. We should leverage both our DOCS and our CODE collections when getting a user query
    b. We can do this by getting K-most relevant chunks from Docs based on user question, and then the K-most relevant Code chunks
    c. From there, we can use these chunks, and pass these to a Cross-Encoder along with the users original query
    d. From this point, we should get a "similarity score" and select only the N most similar scores from this subset of chunks
    e. These N chunks should be the ones that we pass to our LLM
6. **Ingestion From Additional DataSources**
    a. Setup ingestion pipelines for data providers such as **BitBucket and Confluence**